Detailed Functional Requirement Specification

1. Objective

The objective is to enhance the existing strategy application **app1 (Testing.py)** by integrating it with a separate execution and monitoring application (**app2**) for real-time breakout monitoring, trade execution, lifecycle management, and automated exit handling, without modifying any existing strategy logic, calculations, data sources, update mechanisms, or trading rules implemented in the original strategy code.

---

2. Existing System Overview

App1 – Strategy Engine (Testing.py)

The existing strategy engine is responsible for:

- Data collection and updates.
- Breakout candle identification.
- Trade signal generation.
- Stop-loss calculation.
- Target calculation.
- Trailing stop-loss calculation.
- Breakout invalidation logic.
- Backtesting functionality.
- Auto-update functionality.

App2 – Trade Execution & Monitoring Engine

App2 shall be responsible for:

- Real-time market monitoring.
- Breakout crossover detection.
- Order execution.
- Position management.
- Exit monitoring.
- Telegram notifications.
- Fund verification.
- Trade lifecycle management.

---

3. Core Development Principles

The following requirements are mandatory:

3.1 Strategy Preservation

The original strategy code in Testing.py shall remain functionally unchanged.

The following components must remain exactly identical to the existing implementation:

- Strategy rules
- Entry conditions
- Exit conditions
- Breakout identification logic
- Breakout invalidation logic
- Stop-loss calculations
- Target calculations
- Trailing stop-loss calculations
- Data sources
- Data update intervals
- Backtesting logic
- Auto-update logic

3.2 Backtest Execution

During:

- Initial full backtest
- Historical processing
- Data rebuilding

No Telegram alerts shall be generated.

No communication shall be sent to App2.

---

4. Breakout Communication Framework

4.1 Standard Breakouts

Whenever App1 identifies a valid breakout candle, it shall immediately send the following information to App2:

Long Breakout

- Breakout Type = LONG
- Breakout Candle Timestamp
- Entry Price (High + 10)
- Stop-loss Price
- Unique Breakout Identifier

Short Breakout

- Breakout Type = SHORT
- Breakout Candle Timestamp
- Entry Price (Low - 10)
- Stop-loss Price
- Unique Breakout Identifier

4.2 OGBC Handling

OGBC = Original Breakout Candle whose calculated stop-loss is greater than or equal to 90 points.

For such breakouts:

- No breakout alert shall be immediately sent.
- No trade signal shall be generated.
- No crossover monitoring shall begin.

Instead:

- The breakout shall be treated as a virtual breakout.
- Monitoring shall begin only when a valid replacement setup is identified.
- Alerts shall only be sent after replacement candle confirmation.

---

5. Real-Time Monitoring Framework

5.1 Monitoring Trigger

Upon receiving breakout details from App1, App2 shall immediately begin monitoring the breakout using live market prices.

Monitoring shall not wait for candle closure confirmation from App1.

5.2 Live Data Infrastructure

Requirements

A single live NIFTY ticker connection shall be maintained.

The implementation shall:

- Use Kite WebSocket.

- Use the connection architecture implemented in PaperTradeV6.
- Minimize latency.
- Prevent duplicate API calls.

Resource Optimization

If:

- No active breakout monitoring exists

Then:

- Live ticker subscription shall remain inactive.

If:

- One or more breakouts exist

Then:

- One live ticker shall serve all monitoring processes.

No separate ticker connection shall be created per breakout.

---

6. Breakout Invalidation Handling

If App1 invalidates a breakout before crossover:

App1 shall immediately send:

- Breakout Identifier
- Invalidation Status

Upon receipt:

- App2 shall terminate monitoring.
- No trade shall be generated.
- No further actions shall occur.

---

7. Crossover Detection Logic

7.1 Long Breakout

Entry Level:

Entry Price = Breakout Candle High + 10

Crossover Condition:

Live NIFTY Price >= Entry Price

Upon satisfaction:

- Trade Signal generated.
- Trade lifecycle begins.

7.2 Short Breakout

Entry Level:

Entry Price = Breakout Candle Low - 10

Crossover Condition:

Live NIFTY Price <= Entry Price

Upon satisfaction:

- Trade Signal generated.
- Trade lifecycle begins.

---

8. Trade Signal Creation

Upon crossover:

A trade instance shall be created.

Example:

BOT1 = Breakout Trade 1

Stored Information:

- Trade ID
- Breakout ID
- Entry Level
- Stop-loss Level

- Signal Time
- Trade Direction
- Order Tracking Information

---

9. Expiry Selection Logic

Monday / Tuesday

Use:

Next Weekly Expiry

Wednesday / Thursday / Friday

Use:

Nearest Weekly Expiry

Option selection logic shall follow PaperTradeV6 implementation.

---

10. Pre-Trade Fund Verification

Before placing any order:

Available Trading Capital ≥ ₹60,000

If funds are insufficient:

- Reject trade.
- Generate Telegram notification.
- Continue monitoring future signals.

If funds are sufficient:

- Proceed with order placement.

---

11. Option Spread Construction

11.1 Long Breakout

Strategy:

Bull Put Spread

Leg Selection:

- Sell ATM Put
- Buy ATM - 200 Put

Strike Difference:

200 Points

ATM calculation shall follow PaperTradeV6 methodology.

11.2 Short Breakout

Strategy:

Bear Call Spread

Leg Selection:

- Sell ATM Call
- Buy ATM + 200 Call

Strike Difference:

200 Points

ATM calculation shall follow PaperTradeV6 methodology.

---

12. Order Placement Workflow

12.1 Entry Sequence

Order placement sequence must always be:

Step 1: Place Buy Leg

Step 2: Place Sell Leg

This sequence shall never be reversed.

12.2 Order Parameters

For both entry and exit:

- Product Type = Overnight
- Order Type = Market
- Variety = Regular
- Quantity = 65 (1 Lot)

12.3 Execution Validation

After placement:

- Verify Buy Leg execution.
- Verify Sell Leg execution.

If execution fails:

- Retry order placement.
- Send Telegram alert.

---

13. Trade Identification Framework

Each trade shall maintain:

- Unique Trade ID
- Breakout ID
- Entry Timestamp
- Strike Information
- Order IDs
- Execution Time

This information shall be stored for future trade management and exit execution.

---

14. Trade Monitoring Logic

Monitoring shall use:

- Original Entry Level generated by BOT1.
- Original Stop-loss generated by BOT1.

Actual market slippage shall not modify monitoring levels.

---

15. Exit Management

Exit Condition 1 – Stop-Loss Exit

Long Trade

Exit when:

Live Price $\leq$ Stop-loss

Short Trade

Exit when:

Live Price $\geq$ Stop-loss

Immediate spread exit required.

---

16. Trailing Stop-Loss Logic

Long Trade

Stage 1

When:

Price $\geq$ Entry + 140

Then:

Stop-loss = Entry

Stage 2

When:

Price ≥ Entry + 240

Then:

Dynamic trailing activated.

Dynamic Stop-loss:

Latest High - 100

Update Latest High after every completed 15-minute candle.

Exit when:

Price ≤ (Latest High - 100)

Short Trade

Stage 1

When:

Price ≤ Entry - 140

Then:

Stop-loss = Entry

Stage 2

When:

Price ≤ Entry - 240

Then:

Dynamic trailing activated.

Dynamic Stop-loss:

Latest Low + 100

Update Latest Low after every completed 15-minute candle.

Exit when:

Price ≥ (Latest Low + 100)

---

17. Exit Order Execution

Upon exit trigger:

Identify trade using:

- Trade ID
- Order Time
- Strike Mapping

Exit only the legs belonging to that specific trade.

Exit Sequence

1. Buy Leg First
2. Sell Leg Next

Exit Parameters

- Variety = Regular
- Product = Overnight
- Order Type = Market
- Quantity = 65

Validate execution and retry if required.

---

18. Duplicate Protection

The system shall ensure:

- Only one active entry per trade signal.
- No duplicate entry orders.
- No duplicate exit orders.
- No duplicate monitoring instances.

---

19. Multiple Trade Management

The system shall support:

- Multiple breakout monitoring.
- Multiple active trades.
- Multiple spread positions.

Each trade shall operate independently using its own:

- Trade ID
- Entry Level
- Stop-loss
- Strike Mapping
- Order Information

All monitoring shall utilize a single live ticker connection.

---

20. Telegram Notification Framework

Telegram notifications shall be generated for:

Breakout Events

- New breakout detected
- Breakout invalidated

Trade Events

- Trade signal generated
- Order placed
- Order failed
- Order retry
- Trade exited

Account Events

- Fund availability
- Insufficient funds

Error Events

- API failure
- Order execution issues
- Monitoring issues

Lifecycle Events

- Entry completed
- Stop-loss exit
- Trailing stop-loss exit
- Trade closure